Architecture Specification

Program topology, account model, PDA conventions, and the instruction surface of RoundFi

RoundFi
2026-06-23

CONTENTS

> **Derived document.** This is a derivation of `docs/spec/MASTER-SPEC.md` (§4 Protocol → instructions, §11 Deployed state). The Master Spec is the single source of truth; if an ID or schema here disagrees with it, the Master Spec wins. Every program ID, account, and instruction is pinned to the deployed Jun 2026 source (`programs/` , `crates/math/`). Citations point at the source file so this document can be re-verified, not trusted.

1. Program topology

RoundFi is **four Solana programs**. One holds the protocol; one holds the portable reputation; two are interchangeable yield adapters behind a common interface. Keeping reputation in its own program is what makes the score **portable** — it is not entangled with any single pool's lifecycle, and an external consumer can read it without touching pool state.

Program	Role
`roundfi-core`	**All protocol logic** — pool lifecycle, the four vaults, contributions, default settlement, the Escape Valve, the pause circuit breaker.
`roundfi-reputation`	The **on-chain level ladder** and attestation records — score, tier, completed-pools, the identity gate.
`roundfi-yield-mock`	A **devnet** yield adapter (deterministic, for end-to-end testing of the Yield Cascade).
`roundfi-yield-kamino`	The **mainnet-target** yield adapter — the Kamino CPI integration.

1.1 Deployed program IDs

The four programs are deployed on devnet and **share their IDs across clusters** (MASTER-SPEC §11):

Program	ID
`roundfi-core`	`8LVrgxKwKwqjcdq7rUUwWY2zPNk8anpo2JsaR9jTQQjw`
`roundfi-reputation`	`Hpo174C6JTCfiZ6r8VYVQdKxo3LBHaJmMbkgrEkxe9R2`
`roundfi-yield-mock`	`GPTMPgxexhwkhXNovnfrcSsmoWPUhedvKAQfTV2Ef5AQ`
`roundfi-yield-kamino`	`74izMa4WzLuHvtzDLdNzcyygKe5fYwtD95EiWMuzhFdb`

1.2 The yield-adapter interface

`roundfi-core` does not call Kamino directly. The two yield programs are **pluggable adapters** behind one interface: `deposit_idle_to_yield` and `harvest_yield` issue CPIs to whichever adapter the pool is configured against. `roundfi-yield-mock` is the devnet implementation; `roundfi-yield-kamino` is the mainnet target (on-chain CPI shipped, operational reserve pin pending). Swapping adapters does not touch pool or reputation state — the float simply routes through `yield_vault` to a different program.

2. Account and PDA model

2.1 The Pool account

The **Pool** PDA is the spine of a deployment. It is allocated by `create_pool` and carries the lifecycle status (`Forming` / `Active` / `Completed` / `Closed`), the geometry (`members_target` , `cycles_total` , `current_cycle`), the timing (`next_cycle_at` , `cycle_duration`), the split parameters (`escrow_release_bps`), and the Yield-Cascade earmarks (`guarantee_fund_balance` , `lp_distribution_balance`). A core invariant is fixed at allocation: **`cycles_total == members_target`** (`create_pool.rs:123` , SEV-038) — every slot is drawn exactly once.

2.2 The Member account

A **Member** PDA records one wallet's position in one pool: `contributions_paid` , `escrow_balance` , `on_time_count` , `late_count` , `slot_index` , the stake tier, and the `defaulted` flag. This is **per-pool operational state**, distinct from the wallet's portable reputation (§2.5). On an Escape Valve sale the seller's Member PDA is **closed** and the buyer's is **created** carrying the **position-state snapshot** — `contributions_paid` , `escrow_balance` , `on_time_count` , `late_count` , `slot_index` , stake tier — so obligations transfer with the slot while reputation does not.

2.3 The four vaults

Each pool owns **four USDC token accounts**, created by `init_pool_vaults` , all with **PDA authorities**:

Vault	Purpose
`pool_usdc_vault`	The **pool float** — contributions land here net of splits; payouts and seized funds flow here.
`escrow_vault`	Member **stakes** + the escrow slice of each contribution; the vesting / seizure surface.
`solidarity_vault`	The **"Cofre Solidário"** 1%-of-installment reserve; first tranche of any default.
`yield_vault`	Idle USDC parked into the configured yield adapter.

Vault creation is split out of `create_pool` into `init_pool_vaults` because allocating the Pool PDA and all four ATAs in a single instruction would exceed Solana's **stack-depth** budget.

2.4 The seizure / vesting surface

`escrow_vault` and `solidarity_vault`, together with the member stake, are the physical accounts the **Triple Shield** acts on. `settle_default` seizes from them in a fixed order (solidarity → escrow → stake); `release_escrow(checkpoint)` vests stake back out of escrow for an on-time member. The mechanics are specified in `02-technical-whitepaper` §4–§5; here they are listed as the accounts those instructions mutate.

2.5 The reputation accounts (`roundfi-reputation`)

The portable credit signal lives in its own program, in its own PDAs:

- **`ReputationProfile`** — keyed **`[b"reputation", wallet]`**. Holds the wallet's **score**, **tier** (L1–L4), `cycles_completed`, and identity flag. Because it is keyed by **wallet**, it is **non-transferable** and tracks the wallet across every pool — an Escape Valve buyer keeps their own profile and inherits none of the seller's. This is the account an external lender reads to price credit.
- **`IdentityGateConfig`** — the configurable identity gate. Field `required_min_level` governs whether L2/L3 require verification (devnet `0` = open, mainnet `3` = verified-only). The L4 hard floor (`IDENTITY_HARD_FLOOR_LEVEL`) is **independent** of this config — Elite **always** requires `identity_verified`, even with the gate off.

The four-tier ladder, the score schedule, and the Pass-3 attestation schemas that write these accounts are specified in `04-behavioral-reputation-score`.

2.6 Position NFTs

Each pool slot is represented by a **position NFT** (mpl-core). On an Escape Valve buy the NFT is **thawed** → **transferred** → **re-frozen** via mpl-core CPIs, authorized by the slot's `position_authority` PDA. The frozen-by-default state is what binds the NFT to its slot's obligations between trades.

3. The instruction surface

The instruction surface, grouped by lifecycle phase. Every named instruction below appears in MASTER-SPEC §4; nothing is invented here.

3.1 Pool creation and membership

Instruction	Effect
`create_pool`	Allocates the Pool PDA; enforces `cycles_total == members_target` and the Seed Draw viability guard (reverts `PoolNotViable`).
`init_pool_vaults`	Creates the four USDC vault ATAs (split from `create_pool` for stack depth).
`join_pool`	Member joins and locks stake; flips the pool **Active** when `members_joined == members_target`.

3.2 The contribution cycle

Instruction	Effect
`contribute`	Pays an installment; splits it 1% / 25% / 74% into solidarity / escrow / float; emits the on-time, late, or pool-complete attestation.
`claim_payout`	The drawn member claims the carta; advances `current_cycle`, re-arms `next_cycle_at`; flips **Completed** on the final cycle.
`release_escrow`	Vests stake back out of escrow for an on-time member, gated on `on_time_count ≥ checkpoint` (else `EscrowLocked`).

3.3 Default settlement

Instruction	Effect
`settle_default`	Seizes solidarity → escrow → stake into `pool_usdc_vault`; marks `defaulted`; emits `SCHEMA_DEFAULT`. Eligibility gated on the **7-day mainnet / 1-day canary** grace window.

3.4 Yield Cascade

Instruction	Effect
`deposit_idle_to_yield`	Routes idle `pool_usdc_vault` float into `yield_vault` via the adapter CPI.
`harvest_yield`	Realizes yield and runs the 20% fee → guarantee fund → 65% LP → residual waterfall; `min_realized` slippage floor.

3.5 Escape Valve (secondary market)

Instruction	Effect
`escape_valve_list`	**Direct** path: lists a position at a clear `price`, immediately `Active`.
`escape_valve_list_commit`	**Commit-reveal** path: posts `SHA-256(price ‖ salt)`, hiding the price.
`escape_valve_list_reveal`	Reveals `(price, salt)`; arms `buyable_after = now + REVEAL_COOLDOWN_SECS` (30s).
`escape_valve_buy`	Atomically transfers price, swaps the Member PDA (position-state snapshot), moves the NFT, closes the listing; enforces `now ≥ buyable_after` (`ListingNotBuyableYet`).

3.6 Termination and rent reclaim

Instruction	Effect
`close_pool`	Pure terminal `Completed → Closed`; decrements committed TVL, **moves no funds**.
`close_member`	Reclaims a Member PDA's rent to the member (the rent-reclaim ceremony, SEV-039).
`close_pool_vaults`	Sweeps residual USDC → treasury, closes the vault ATAs + Pool PDA, returns rent → authority.

`close_pool` only flips the status; the **true** end of a pool's life is the rent-reclaim ceremony — `close_member` × N (rent → members) followed by `close_pool_vaults` (residual → treasury, accounts closed, rent → authority).

3.7 Reputation and identity (`roundfi-reputation`)

Instruction	Effect
`promote_level`	**Permissionless**: advances a wallet to the highest tier whose score **and** completed-pools thresholds are both met (`resolve_level`), then applies `cap_level_for_identity`.

4. The pause carve-out

`pause(bool)` (authority-gated) sets `config.paused`. The gate is implemented as a `!config.paused` **account-validation constraint** on the shared `config` account, so paused instructions revert with `ProtocolPaused` **before** any handler logic runs.

13 instructions are gated while paused — including `create_pool`, `join_pool`, `contribute`, `claim_payout`, `release_escrow`, `deposit_idle_to_yield`, `harvest_yield`, and **both Escape Valve paths**.

`settle_default` is the single deliberate carve-out. A default already in flight must remain settleable while the rest of the protocol is frozen, so that a pause cannot trap a pool with an

unrecoverable delinquent member. Structurally this means `settle_default` does **not** carry the `!config.paused` constraint that the other thirteen do — the exemption is enforced by the **absence** of the gate on exactly one instruction.

5. Validated topology

Seven of the eight capability areas were exercised **end-to-end on devnet** on 2026-06-12 with real USDC; the eighth (`settle_default`) is armed and time-gated. The instruction surface above is therefore not aspirational — the lifecycle (`Ga2RwgSk…`), the Yield Cascade and both Escape Valve paths (`4SZCKeQL…`), the pause gate, escrow vesting, and the full rent-reclaim ceremony have all run on the deployed programs. Transaction signatures and the reproducible runbook live in `docs/operations/v52-devnet-runbook.md` .

Cross-references: the end-to-end protocol mechanics these accounts and instructions implement → `02-technical-whitepaper` ; the reputation engine, ladder, and Pass-3 schemas behind `roundfi-reputation` → `04-behavioral-reputation-score` . The canonical instruction list and program IDs live in MASTER-SPEC §4 and §11.